ECE 350: Real-Time Operating Systems Winter 2027

Lecture 26 — Flash Storage, SSDs, and Log-Structured File Systems

Jeff Zarnett *2026-07-12*

Solid Snake? No, Solid State

Just as the previous topic started off with talking about the physical construction of a magnetic hard drive, let's start by talking about solid state drives work. You might have even had a hard drive that died on me in 2014 in your hands, to allow you to explore its mechanical components. The SSD version of this does not involve quite as much hands-on experience with the device, for two reasons. For one, I don't have a dead SSD device to show you. The other is that it's much less interesting when you hold it in your hands. It looks like a RAM stick, honestly.

Once again, solid state drives are called that because they have no moving parts. That means the question is not the scheduling algorithm for reads and writes; it's a uniform access time device and therefore it can be FCFS or perhaps that with some variation to reflect priorities. And yet, SSDs have some nuances to them that make this more interesting than "hard drives, but faster and uniform".

SSDs are typically built using NAND Flash memory. The details of that are interesting, but perhaps too low-level for us to think about. The important thing that matters for the discussion here, however, is that SSDs are write-limited devices. That is, they have a maximum number of write cycles and after that, they're finished. Should I have said cooked? They're cooked.

This is profoundly weird as compared to everything else in the system. Yes, we understand that nothing is forever, or things can break or get damaged. But for the other components of the system (CPU, RAM, magnetic drive, etc.) there's no maximum number of things. Can you imagine? Sorry, your CPU has reached its instruction execution maximum and cannot do any more, guess you have to buy a new one. Wait. I'm wrong. Printers from predatory companies do this and I *hate* it. Don't buy a printer that does this. The toner cartridge is finished when it is completely empty, not when HP (yes, I'm naming names) has decided you should have to buy a new one. In perhaps a less infuriating example, Lithium-Ion batteries also have a lifecycle that's measured in the number of discharge cycles. And that is a limitation imposed by the chemistry of the battery, not arbitrary manufacturer "shareholder value" calculations.

Why do the NAND flash chips break down with writing? It's the physics of the thing: the high field and current stress from writing to the chip breaks down the oxide which eventually renders a given cell useless [Kha09]. Once that has happened to enough cells, the device is no longer capable of writing data. Maybe you can read it. The device is rated for N cycles for each of the cells, but exactly how many depends on the specific drive and its technology. Under normal use, you will probably never encounter this: your laptop, phone, or other device will likely be retired for other reasons before you ever get close to exhausting the ability to write to disk.

How big is N cycles, though? It's typically something like 10 000 cycles for an individual cell [ADAD23]. You might think that's a lot or you might think it's very little. Let's say it's your laptop. Lots of files on the laptop are written once or rarely. Even these lecture notes – yes, I write them and I revise them from time to time but once the course material is written and all the typos have been fixed (haha, as if), will it change again? Not for a long time. But we learned about the swap file earlier: the swap file is read and written a lot! 10 000 writes the swap file can happen in a day, easily, so then what?

This has led us to something important: when the SSD writes to the same file, that data might not be in the same place(s) on disk anymore. In fact, even if we wanted to, it would be slow: to write data to a particular block, we have to erase what's there and then write the new block. But erasing is slow. So the solution is to write to a new location and then we can go back and erase the old location at a later date. This has some similarities with page faults: if there is an empty frame in memory, it means we can do the read into that frame immediately and evict another page from a frame later, asynchronously.

The consequences of this idea that physical locations move sometimes immediately leads to four things that matter in the implementation: (1) there has to be some indirection so when the OS says "read block 9876" we map that to a physical location; (2) some sort of wear-levelling to distribute the writes to preserve write-capacity, (3) a form of garbage collection, and (4) there are additional writes that are overhead. There's more to SSDs than those things, but let's do these and come back to the rest later.

Flash Translation Layer. The level of indirection is called FTL (Flash Translation Layer, not Faster Than Light). This layer needs to take a request and determine its physical location. We will examine a few different strategies for this.

Wear-Levelling. Given that every block is as fast as any other, it truly does not matter from the perspective of access speed where the data is. That means that it is possible for the device to always choose to write things to the available area that has the fewest number of write cycles accumulated.

Recognizing that a lot of files don't change very often, it's possible that you have some areas of the disk that are allocated but written to only once. A thorough wear-levelling program would move those things around periodically to ensure that we are really wearing things down as evenly as possible.

Garbage Collection. Just like in languages that do this for memory, garbage collection is a period process of cleaning up things that are no longer referenced/needed. The process requires a way of keeping track of what is needed and what is not; with that information, the device can iterate over the pages that we have and reclaim the space to make it available again. A more complex algorithm can reduce the amount of work done, but the basic idea does not change. Part of the motivation for doing it with garbage collection rather than disposing of the data immediately is that the garbage collection process can run at any convenient (i.e., low-utilization) time rather than synchronously when the drive is under heavy use.

Write Amplification. Write Amplification is the term that is used to express how much additional writing takes place above the logical write that the OS or application asked for. For example, if the write is a 4 KB chunk of a file, the bytes written within the flash varies. How much extra depends on the workload and thee hardware of the SSD device but it can be 1.2 - 3.0 times (so a 4 KB write results in, on average, 4.8 to 12 KB changed on the drive [HEH+09]. Knowing that every write is consuming some of the limited lifespan of the chips, it almost goes without saying that write amplification is undesirable and something that we'll aim to minimize.

FTL Organization

Before we get into the details here, it's important to note that the word "block" is used to refer to a unit of physical storage and does not necessarily have the same size as a page. This does mean we sometimes have device characteristics like what Wikipedia shows where a 4 KB page is the logical unit for a page but if we do an erase operation we end up erasing 256 KB. Exact numbers will, of course, vary based on the device purchased.

The simplest, and less-than-ideal, approach to FTL mapping is directly mapped: a read of logical page N is mapped directly to a physical page N and a write is done by reading the whole block with the page in it, erase it, and then overwrite that location with the new version [ADAD23]. From what we know already, it is clear why this is bad: it's slow to put the erase action on the critical path for writing the data, and it also wears out that particular location faster. So this approach is clearly not going to do it.

The current approach used by SSDs is called *log-structured*. This means that when we write a logical block N, it just gets added to the next free space in the currently-being-written block [ADAD23]. This requires, obviously, some mapping to exist that translates a logical location to its physical one. The mapping has to be stored in stable storage, otherwise powering the system down means losing all information about what is where, which would be contrary to the idea of the device as persistent storage.

Still, the mapping table is potentially a large amount of data; for every one 1 TB of storage where pages are 4 KB, it costs 1 GB of storage just for the mapping table [ADAD23]. That actually does not seem so bad in some absolute

sense – 1/1000 of the storage for administration seems like a pretty reasonable amount of overhead. The linked source considers this to be a huge amount of overhead and goes through a couple of techniques to reduce the amount of entries in this table; but it is not that interesting so we will skip it.

This approach does lead to garbage, in the sense of garbage collection mentioned previously. When writes are treated as append, there is a need to go back and clean up. If a whole block is no-longer valid data, then it is simple to erase it and consider it free. If a part of the block is still needed, though, it would be a problem to erase the whole thing. And yet, we would not be happy with the idea of wasting 252 KB just because one valid page remains in the block. The solution to that is a sort of compaction: if there is garbage in this block, write the valid sections into a new location and then the whole block can be erased.

Moving things around to make the data compact does help with wear-levelling by ensuring that data can move to new locations periodically, but it also results in more writes to the device that consume its limited lifespan...

TRIM

References

[ADAD23] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 1.10 edition, November 2023.

[HEH+09] Xiao-Yu Hu, Evangelos Eleftheriou, Robert Haas, Ilias Iliadis, and Roman Pletka. Write amplification analysis in flash-based solid state drives. In *Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference*, SYSTOR '09, New York, NY, USA, 2009. Association for Computing Machinery.

[Kha09] Faraz I. Khan. Endurance characterization and improvement of floating gate semiconductor memory devices. 2009.